

TASK 5

API Testing and Validation using Postman/Curl

Testing and Validation Report for the CIFAR-10 Flask Deep Learning Prediction API

Detail	Value
Task	Task 5: API Testing and Validation using Postman/Curl
API under test	CIFAR-10 Deep Learning Prediction API (Flask, developed in Task 4)
Postman collection	Task 5 - CIFAR-10 API Testing
Student name	Sharon Varghese Thoppil
Programme / Course	Deep Learning from Production to Deployment
Institution	B.k Birla College
Roll no	41
Github link	https://github.com/SharonVargheseThoppil/L-T-Task-5/tree/main

Table of Contents

1. Introduction

2. Objective

3. Tools and Technologies Used

4. API Under Test

- 4.1 Endpoint summary
- 4.2 Root endpoint response
- 4.3 Prediction endpoint input
- 4.4 Prediction endpoint success response
- 4.5 Observed error-handling behaviour

5. Postman Collection

- 5.1 Requests defined in the exported collection
- 5.2 Test script present in the collection
- 5.3 Discrepancies between the collection export and the screenshots

6. Test Cases

- 6.1 Test case definitions
- 6.2 Validation criteria and observed outcomes

7. Positive Testing

- 7.1 TC01 — API availability
- 7.2 TC02 — Valid prediction
- 7.3 TC03 — JSON validation request

8. JSON Response Validation

- 8.1 Automated validation
- 8.2 Manual structural validation

9. Negative Testing

- 9.1 TC04 — Missing input
- 9.2 TC05 — Invalid input type
- 9.3 TC06 — Empty input
- 9.4 TC07 — Invalid endpoint

10. Curl Testing

- 10.1 Root endpoint via Curl
- 10.2 Prediction endpoint via Curl

11. Screenshots and Test Evidence

12. Response Logs

- 12.1 Postman response metadata
- 12.2 Flask access log

13. Test Results Summary

14. Observations

15. Conclusion

16. References

17. Source Code

1. Introduction

Task 4 of this series produced a Flask-based deep learning prediction service that exposes a trained CIFAR-10 convolutional neural network over HTTP. Once a model is wrapped in a web API, correctness of the model alone is no longer sufficient: the service must also accept requests in the expected format, return well-formed JSON, and behave predictably when a client sends something unexpected. Task 5 therefore shifts the focus from model development to service verification.

API testing and validation is the systematic process of sending controlled requests to a running service and comparing the observed response against a defined expectation. In the context of a deep learning prediction API, three properties matter in particular:

- Availability — the service starts, loads the trained model, and responds on its advertised address.
- Contract correctness — a valid prediction request returns a JSON document whose structure and field names are stable enough for a client application to consume.
- Robustness — malformed, missing, or unsupported input produces a controlled, documented error response instead of an unhandled crash or an ambiguous reply.

This report documents the testing performed against the Task 4 API using Postman as the primary functional testing client and Curl as an independent command-line cross-check. All endpoints, request configurations, status codes, response times, JSON structures and error messages reproduced in this report are taken directly from the supplied Postman collection export and from the eleven screenshots supplied as testing evidence. Where the two sources of evidence disagree, or where a detail could not be recovered from the evidence, this is stated explicitly rather than filled in by assumption.

2. Objective

To validate and test deep learning prediction APIs using industry-standard testing tools.

The approach prescribed for this task, and followed in the testing documented here, is to:

- Send prediction requests.
- Validate JSON responses.
- Perform negative testing.
- Verify error handling mechanisms.
- Document test results.

3. Tools and Technologies Used

Tool / Technology	Role in this task	Evidence in the supplied files
Postman	Primary API testing client: request construction, form-data file upload, response inspection and test scripting.	Postman collection export "Task 5 - CIFAR-10 API Testing"; Postman UI visible in Figures 1, 2 and 4–9 (running as the Postman extension inside Visual Studio Code).

Tool / Technology	Role in this task	Evidence in the supplied files
Curl	Independent command-line verification of the same endpoints.	Figures 10 and 11 show curl invocations from a PowerShell session.
Flask (Werkzeug development server)	Hosts the CIFAR-10 prediction API under test.	Server started with "python app.py"; response header in Figure 10 reports Server: Werkzeug/3.1.8 Python/3.12.8.
Python 3.12.8	Runtime of the API being tested.	Reported in the Server response header (Figure 10) and by the "(venv) ... python app.py" prompt in the integrated terminal.
TensorFlow / Keras	Provides the trained model loaded by the API.	Startup log in Figure 1 shows TensorFlow warnings and "Model loaded successfully."; the root endpoint reports the model file cifar10_cnn_model.keras.
JSON	Response and error payload format for every endpoint tested.	All responses in Figures 1, 2 and 4–11 are rendered as JSON; Figure 10 reports Content-Type: application/json.
Visual Studio Code	Development environment hosting both the Postman extension and the terminal running the API.	Visible in Figures 1, 4, 6, 7, 8 and 9.
PowerShell	Shell used for the Curl tests.	Prompt "(venv) PS C:\Users\Varghese Thoppil\Task_4_Flask_API>" in Figures 10 and 11.

4. API Under Test

The service under test is the Flask deep learning prediction API produced in Task 4. It was run locally from the project directory Task_4_Flask_API and served on the loopback address http://127.0.0.1:5000. Two endpoints are evidenced in the testing material.

4.1 Endpoint summary

Endpoint	Method	Request type	Purpose as evidenced
/	GET	No body / no parameters	Service metadata and health check. Returns the API description, the prediction endpoint, the accepted method and the loaded model filename.
/predict	POST	multipart/form-data with a file field named image	Accepts an image file and returns the CIFAR-10 classification result with per-class probabilities.

4.2 Root endpoint response

The GET / response observed in Postman (Figure 1) and reproduced identically by Curl (Figure 10) is:

```
{
  "endpoint": "/predict",
  "message": "CIFAR-10 Deep Learning Prediction API",
  "method": "POST",
  "model": "cifar10_cnn_model.keras",
  "status": "API is running"
}
```

This response is self-documenting: it tells a client which path to call, which HTTP method to use, and which model file is loaded. The Curl output additionally reports Content-Length: 171 and Content-Type: application/json.

4.3 Prediction endpoint input

The prediction endpoint expects a single input field. Its name is fixed by the API and is confirmed from three independent places in the evidence:

- The Postman collection defines a form-data key named image of type file for TC02 and TC03.
- The Curl command in Figure 11 uses -F "image=@...", i.e. the same field name.
- The API's own error message names it: "Please upload an image using the 'image' field."

The file used for the valid-prediction tests is test_image.jpg, located at C:\Users\Varghese Thoppil\Task_4_Flask_API\test\sample_images\test_image.jpg. The image itself was supplied with the evidence and is shown in Figure 3.

4.4 Prediction endpoint success response

A successful prediction returns HTTP 200 with a JSON object containing five keys. The complete response captured in Figure 5 is:

```
{
  "class_index": 5,
  "confidence": 99.99,
  "prediction": "dog",
  "probabilities": {
    "airplane": 1.986864232167429e-12,
    "automobile": 9.628090527266913e-15,
    "bird": 1.2026609510940034e-05,
    "cat": 8.43729212647304e-05,
    "deer": 8.452229888211704e-11,
    "dog": 0.9999029636383057,
    "frog": 3.92518274827669e-11,
    "horse": 5.793619948235573e-07,
    "ship": 2.0803866103229728e-15,
    "truck": 2.144675767274534e-13
  },
  "success": true
}
```

The probabilities object enumerates all ten CIFAR-10 classes in the canonical order airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck. The reported class_index of 5 is consistent with dog occupying index 5 in that ordering, and confidence (99.99) is consistent with the dog probability of 0.9999029636383057 expressed as a percentage rounded to two decimal places. These are internal-consistency observations made from the response itself; the API source code was not supplied with this task and was not inspected.

4.5 Observed error-handling behaviour

Three distinct error responses were observed. Each is a JSON object, so error handling is consistent with the success contract in format, though not in field names.

HTTP status	Triggering condition observed	Response body observed
400 Bad Request	No image file present in the request (TC04 and TC06).	{ "error": "No image provided.", "message": "Please upload an image using the 'image' field." }
500 Internal Server Error	A non-image file supplied in the image field (TC05).	{ "details": "cannot identify image file <tempfile.SpooledTemporaryFile object at 0x000001926EF62D70>", "error": "Prediction failed.", "success": false }
404 Not Found	Request to an undefined route (TC07).	{ "error": "Endpoint not found.", "message": "Please check the API URL." }

The 400 and 404 responses use the key pair error/message, whereas the 500 response uses details/error/success. The success key appears in the 200 and 500 responses but not in the 400 or 404 responses. This inconsistency is recorded as an observation in Section 16; it is visible directly in the evidence and is not a fault introduced by the testing.

5. Postman Collection

The supplied export file is Task 5 - CIFAR-10 API Testing.postman_collection.json, conforming to Postman Collection Schema v2.1.0. Its collection name is "Task 5 - CIFAR-10 API Testing" and its identifier is 245b12f3-7691-4d93-a004-baa86b084177. The collection contains no folders; all seven requests sit at the top level of the collection and are named after their test case identifiers.

5.1 Requests defined in the exported collection

The table below records exactly what the export file contains for each request. Several requests were saved without their URL and body, and this is reported as it stands.

Request name	Method	URL in the export file	Body / scripts in the export file
TC01 - API Availability	GET	Empty ("raw": "")	No body, no headers, no test scripts.
TC02 - Valid Prediction	POST	http://127.0.0.1:5000/predict	form-data with key image of type file, src = c:\Users\Varghese Thoppil\Task_4_Flask_API\test\sample_images\test_image.jpg. No test scripts.
TC03 - JSON Validation	POST	http://127.0.0.1:5000/predict	Same form-data file upload as TC02, plus a post-response test script containing two pm.test assertions.
TC04 - Missing Input	POST	Empty ("raw": "")	No body, no headers, no test scripts.
TC05 - Invalid Input	POST	Empty ("raw": "")	No body, no headers, no test scripts.
TC06 - Empty Input	POST	Empty ("raw": "")	No body, no headers, no test scripts.

Request name	Method	URL in the export file	Body / scripts in the export file
TC07 - Invalid Endpoint	GET	Empty ("raw": "")	No body, no headers, no test scripts.

No collection-level variables, environments, authorisation settings or pre-request scripts are defined in the export. Every request in the export has an empty header array, and the "No Environment" selector is visible in the Postman UI in Figures 1, 4 and 6–9, confirming that no environment was active during testing.

5.2 Test script present in the collection

Exactly one request, TC03 - JSON Validation, carries a post-response script. Its full content is:

```
pm.test("Status code is 200", function () {
  pm.response.to.have.status(200);
});

pm.test("Response is JSON", function () {
  pm.response.to.be.json;
});
```

The first assertion validates the HTTP status line; the second validates that the response body parses as JSON and is served with a JSON content type. No assertions on individual response fields (for example prediction or confidence) are defined in the collection.

5.3 Discrepancies between the collection export and the screenshots

Cross-checking the export against the screenshots reveals several differences. They are listed here rather than resolved silently, and the verifiable evidence is identified in each case.

#	Discrepancy	Assessment
D1	TC01, TC04, TC05, TC06 and TC07 have an empty URL in the export file, but the screenshots show each of them with a fully populated URL and, for TC04 and TC05, a populated request configuration.	The screenshots are the verifiable evidence for what was actually sent, because they show both the request and the resulting response together with the matching Flask server log line. The export appears to have been saved before these requests were persisted, or the requests were sent without being saved.
D2	The export records no request body for TC04, TC05 and TC06, yet Figure 6 shows TC04 with an enabled query parameter image, and Figure 7 shows TC05 with form-data image = d:\hi.txt.	Same as D1: the screenshots are used as the record of the request configuration actually exercised.
D3	Figure 4 shows TC03 with Test Results (2/2), consistent with the two assertions in the export. Figure 5 shows a request with Test Results (1/1).	Unresolved. Figure 5 is cropped to the response pane and does not show the request name, so it cannot be attributed to a specific test case with certainty. It is most consistent with an earlier state of TC03 carrying a single assertion. The 2/2 result in Figure 4 is the one that matches the supplied collection.
D4	The sidebar contents differ between screenshots: Figure 6 lists only TC01, TC02 and TC04; Figure 7 adds TC05; Figure 8 adds TC06; Figure 9 adds TC07; only Figure 4 lists all seven including TC03.	Consistent with the screenshots having been captured progressively as the collection was built, rather than in a single final run. It is not in itself a defect.
D5	Figure 4 also shows a second, empty collection named "New Collection" in the sidebar.	An empty artefact of the workspace; it contains no requests and is unrelated to the tests reported here.

#	Discrepancy	Assessment
D6	The Curl tests in Figures 10 and 11 have no counterpart in the Postman collection.	Expected — Curl testing was performed directly from the shell and is documented in Section 10.

6. Test Cases

Seven test cases are evidenced, TC01 to TC07. The naming and numbering are taken from the Postman collection. It should be noted that, apart from the two assertions in TC03, no expected results were formally declared anywhere in the supplied files. The "validation criterion" column below therefore states the criterion applied during this analysis, derived from the API's documented behaviour and from standard HTTP semantics. These criteria are presented as analysis-time validation criteria and are not claimed to have been specified in advance.

6.1 Test case definitions

ID	Description	Method	Endpoint	Input supplied
TC01	API availability / health check	GET	http://127.0.0.1:5000/	None
TC02	Valid prediction request	POST	http://127.0.0.1:5000/predict	form-data, image = test_image.jpg
TC03	JSON response validation with Postman assertions	POST	http://127.0.0.1:5000/predict	form-data, image = test_image.jpg
TC04	Missing input (negative)	POST	http://127.0.0.1:5000/predict?image=	Query parameter image with an empty value; no file part
TC05	Invalid input type (negative)	POST	http://127.0.0.1:5000/predict	form-data, image = d:\hi.txt (a text file, not an image)
TC06	Empty input (negative)	POST	http://127.0.0.1:5000/predict	No parameters and no body
TC07	Invalid endpoint (negative)	GET	http://127.0.0.1:5000/invalid	None

6.2 Validation criteria and observed outcomes

ID	Validation criterion applied	Actual result observed	Status	Observation
TC01	HTTP 200 with a JSON body identifying the service.	200 OK, 12 ms, 337 B; JSON reporting status "API is running" and model cifar10_cnn_model.keras.	Pass	Service reachable and self-describing.
TC02	HTTP 200 with a JSON prediction payload.	200 OK, 1108 ms, 637 B; JSON with class_index 5, prediction "dog", confidence 99.99.	Pass	Response time markedly higher than later calls; see Section 14.
TC03	The two pm.test assertions in the collection must pass.	200 OK, 130 ms, 637 B; Test Results 2/2 in Figure 4.	Pass	Identical payload to TC02 at roughly one-eighth the latency.
TC04	A request with no image file must be rejected with a client-error status and an explanatory JSON message.	400 Bad Request, 7 ms, 275 B; {"error": "No image provided.", ...}.	Pass	Query parameter is correctly not accepted as a file input.

ID	Validation criterion applied	Actual result observed	Status	Observation
TC05	A non-image file should be rejected with a controlled error response.	500 Internal Server Error, 389 ms, 345 B; {"error": "Prediction failed.", "success": false} with a details field.	Partial	Error is caught and returned as JSON, but the status code is a server-error class where a 400-class response would be expected for bad client input. See Section 9.2.
TC06	A request with no parameters and no body must be rejected with a client-error status and an explanatory JSON message.	400 Bad Request, 6 ms, 275 B; identical body to TC04.	Pass	Consistent with TC04.
TC07	An undefined route must return 404 with a JSON error rather than an HTML error page.	404 Not Found, 10 ms, 251 B; {"error": "Endpoint not found.", "message": "Please check the API URL."}.	Pass	Custom JSON 404 handler is in place.

Status values used above: Pass = the observed behaviour satisfies the stated criterion; Partial = the request was handled without crashing the service and returned a structured JSON error, but the HTTP status class does not match the nature of the fault.

7. Positive Testing

Positive testing covers the requests that supply valid input and are expected to succeed: TC01, TC02 and TC03.

7.1 TC01 — API availability

A GET request was sent to `http://127.0.0.1:5000/` with no parameters, no headers configured by the tester and no body. The service returned HTTP 200 OK in 12 ms with a 337-byte response. The JSON body is reproduced in Section 4.2. The integrated terminal in the same screenshot shows the Flask startup sequence, including "Model loaded successfully.", the debugger PIN, and two access log lines recording GET / with status 200 at 14:34:29 and 14:50:44.

Validation performed: the status line, the presence of the status field with the value "API is running", and the presence of the endpoint, method and model fields identifying how the prediction endpoint should be called.

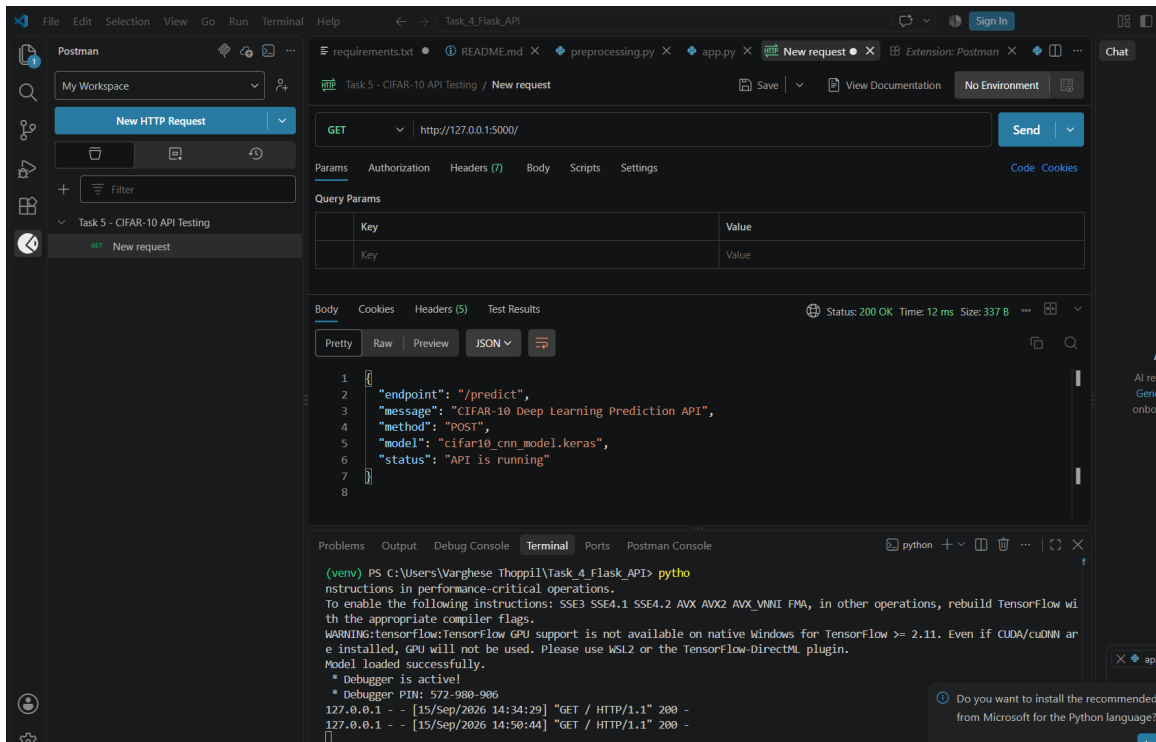


Figure 1 — TC01: GET `http://127.0.0.1:5000/` returning 200 OK in 12 ms, with the Flask startup log and access log visible in the integrated terminal.

7.2 TC02 — Valid prediction

A POST request was sent to `http://127.0.0.1:5000/predict` with a multipart/form-data body containing a single field, image, populated with the file `test_image.jpg`. The service returned HTTP 200 OK in 1108 ms with a 637-byte JSON response reporting `class_index` 5, confidence 99.99 and prediction "dog". The terminal pane in the same screenshot records the matching server-side log line: `POST /predict HTTP/1.1" 200` at 15:02:58.

Validation performed: the status line, the presence and types of the `class_index`, confidence and prediction fields, and agreement between the response size reported by Postman and the payload shown.

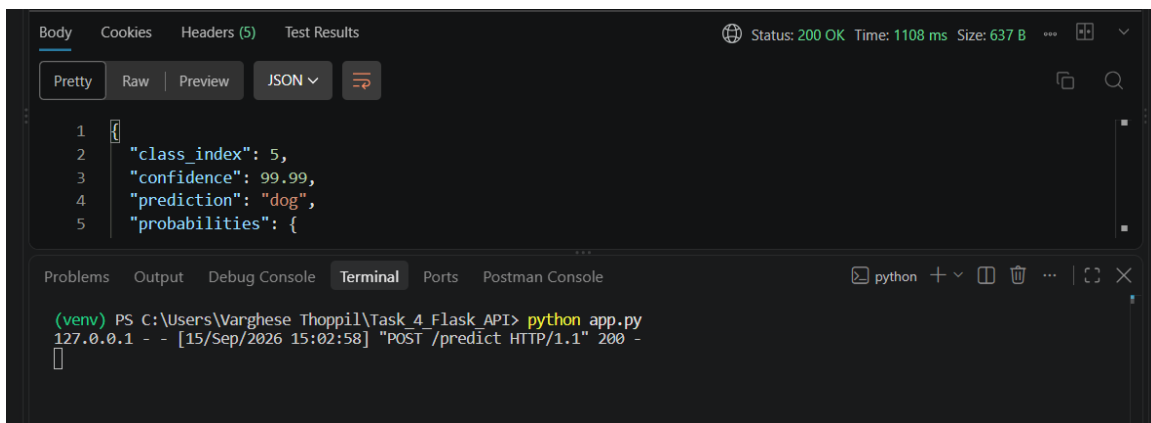


Figure 2 — TC02: 200 OK in 1108 ms for the valid prediction request, with the corresponding Flask access log entry below.

The response pane in Figure 2 is scrolled to the first five lines of the body; the complete payload for this request shape is captured in Figure 5 and reproduced in Section 4.4.



Figure 3 — The input image test_image.jpg used for TC02, TC03 and the Curl prediction test. The API classified this image as "dog".

7.3 TC03 — JSON validation request

TC03 repeats the TC02 request and adds the two post-response assertions listed in Section 5.2. The service returned HTTP 200 OK in 130 ms with the same 637-byte payload, and Postman reported Test Results (2/2), meaning both assertions passed. Detailed discussion of this test appears in Section 8.

8. JSON Response Validation

Response validation was carried out at two levels: automated assertions executed by Postman, and manual structural inspection of the rendered payload.

8.1 Automated validation

The two pm.test assertions attached to TC03 check the status code and the JSON nature of the response. Both passed, as shown by the "Test Results (2/2)" label in Figure 4 below. The 130 ms response time and 637-byte size shown alongside match the successful prediction payload.

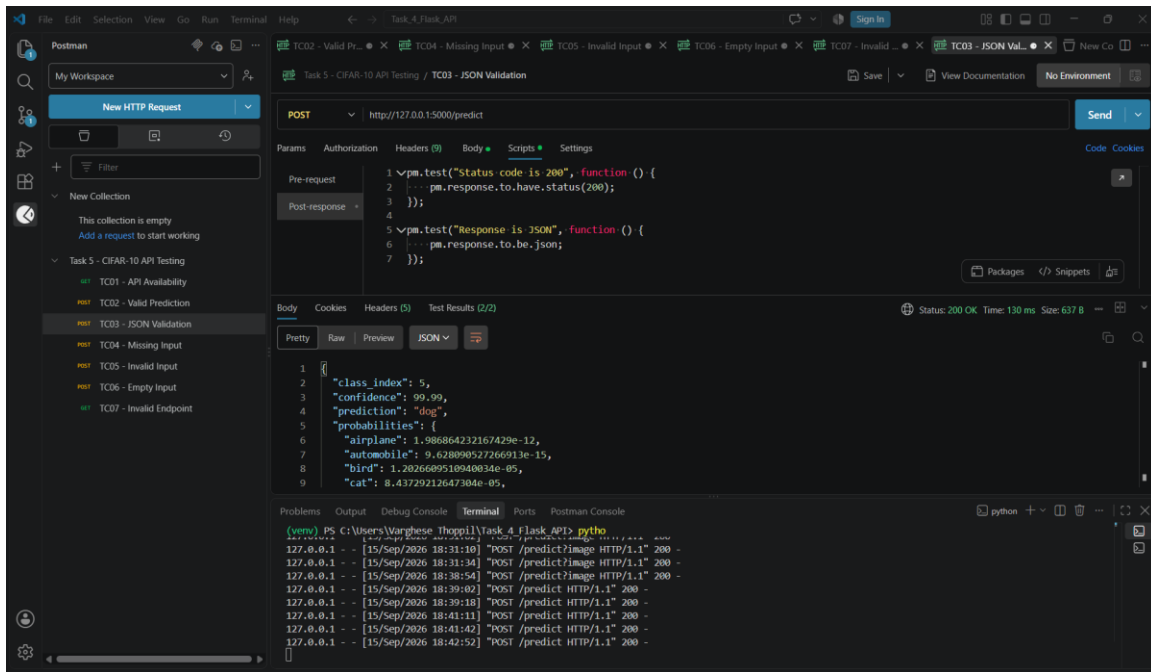


Figure 4 — TC03: the Postman post-response script (upper pane), the 200 OK / 130 ms / 637 B result with Test Results (2/2), and the accumulated Flask access log (lower pane).

Note on scope: these assertions validate the transport-level contract — that the call succeeded and that the body is JSON. They do not assert on the values or presence of individual fields. Field-level validation in this task was therefore performed manually, as described below.

8.2 Manual structural validation

The full response body was inspected in Postman's JSON view (Figure 5). The following checks were carried out against the payload reproduced in Section 4.4:

- Top-level structure: five keys are present — `class_index`, `confidence`, `prediction`, `probabilities` and `success`.
- Types: `class_index` is an integer, `confidence` is a number, `prediction` is a string, `probabilities` is a nested object and `success` is a boolean.
- Class coverage: `probabilities` contains exactly ten entries, one per CIFAR-10 class.
- Internal consistency: the class named in `prediction` ("dog") is the class with the highest probability (0.9999029636383057), and its index in the canonical CIFAR-10 class ordering (5) matches `class_index`.
- Confidence consistency: 0.9999029636383057 expressed as a percentage and rounded to two decimals gives 99.99, matching the `confidence` field.
- Success flag: `success` is true for the successful prediction.

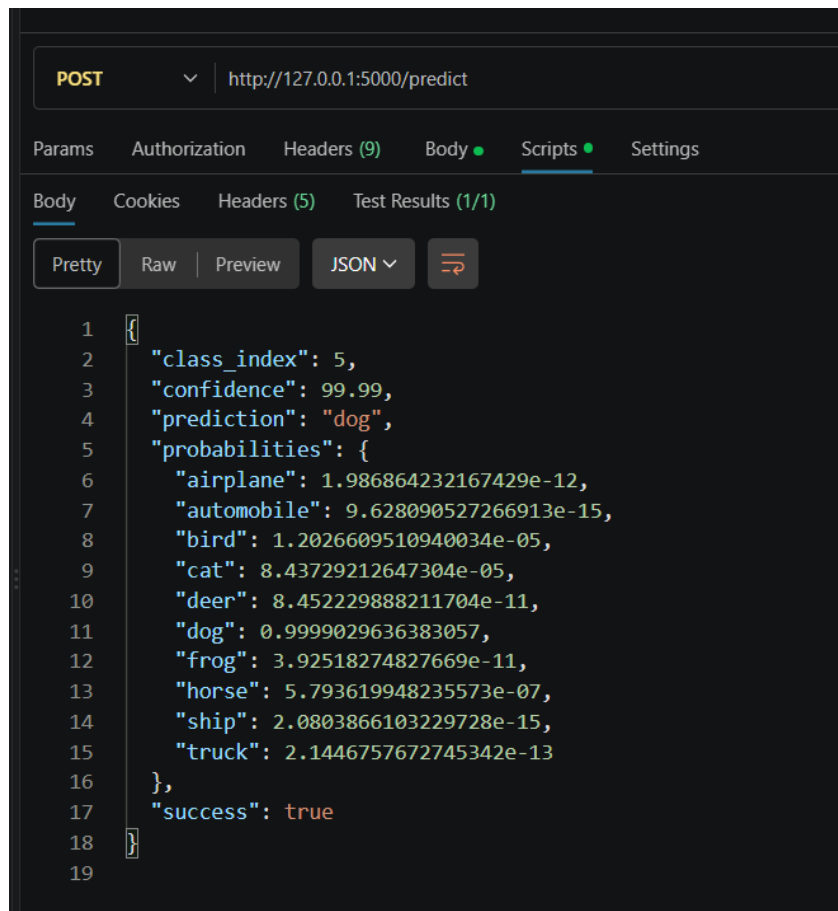


Figure 5 — Complete JSON prediction response showing all five top-level keys and the ten per-class probabilities. The Test Results label on this capture reads (1/1); see discrepancy D3 in Section 5.3.

Cross-tool validation: the identical payload, to the full precision of every probability value, was returned by the Curl request in Figure 11. Two independent clients therefore produced byte-equivalent prediction output for the same input file, which supports the conclusion that the response is deterministic and not client-dependent.

9. Negative Testing

Four negative test cases are evidenced: TC04, TC05, TC06 and TC07. Each is described in terms of what was intentionally made incorrect, what the API returned, and whether the error handling behaved acceptably.

9.1 TC04 — Missing input

What was intentionally incorrect: the request was sent to /predict with an enabled query parameter named image carrying no value, producing the URL `http://127.0.0.1:5000/predict?image=`. No multipart file part was attached. This tests whether the API mistakes a URL query parameter for the required file field.

Observed response: HTTP 400 Bad Request in 7 ms, 275 bytes:

```
{
  "error": "No image provided.",
  "message": "Please upload an image using the 'image' field."
}
```

The Flask access log in the same screenshot records the request as POST /predict?image= HTTP/1.1" 400 at 16:28:30, confirming server-side that the request was received and rejected with the status shown in the client.

Assessment: error handling worked as observed. The API correctly distinguishes a query parameter from a multipart file field, returns the appropriate client-error status, and supplies a remediation message naming the expected field.

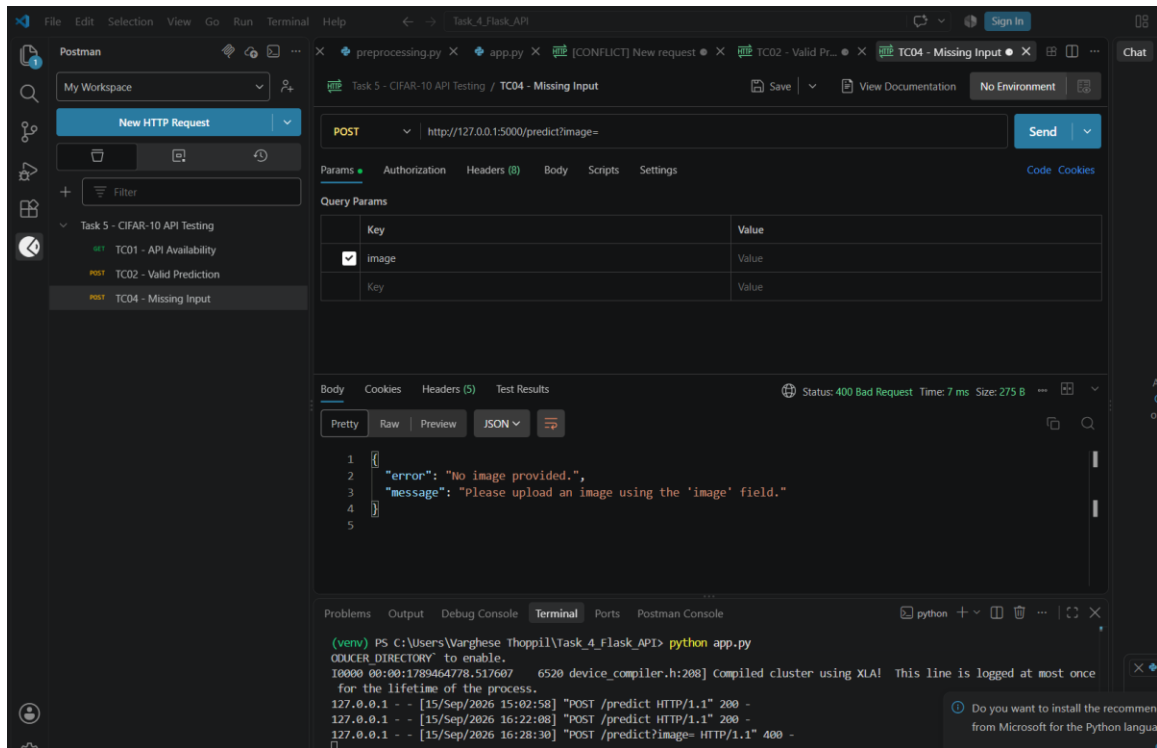


Figure 6 — TC04: POST to /predict?image= returning 400 Bad Request in 7 ms with the "No image provided." JSON error, and the matching 400 entry in the Flask access log.

9.2 TC05 — Invalid input type

What was intentionally incorrect: the form-data field image was populated with d:\hi.txt, a plain text file rather than an image. This tests the API's handling of a correctly structured request carrying unusable content.

Observed response: HTTP 500 Internal Server Error in 389 ms, 345 bytes:

```
{
  "details": "cannot identify image file <tempfile.SpooledTemporaryFile object at 0x000001926EF62D70>",
  "error": "Prediction failed.",
  "success": false
}
```

The access log records POST /predict HTTP/1.1" 500 at 16:57:14.

Assessment: error handling worked partially. Positively, the failure was caught rather than propagated as an unhandled exception: the service returned a structured JSON body with success set to false, remained running, and continued to serve subsequent requests (TC06 and TC07 were executed afterwards against the same process). Negatively, two aspects are worth recording as findings:

- The status code is 500, which by HTTP convention signals a server-side fault. The fault here originates in the client-supplied content, so a 400-class status such as 400 Bad Request or 415 Unsupported Media Type would communicate the cause more accurately.
- The details field exposes an internal Python exception message, including the repr of a SpooledTemporaryFile object and its memory address. This is useful during development but leaks implementation detail that would normally be suppressed in a production deployment.

The 389 ms response time, against 6–7 ms for the two 400 responses, indicates that this request travelled further into the processing pipeline before failing — consistent with the error arising at the point of image decoding rather than at the input-presence check.

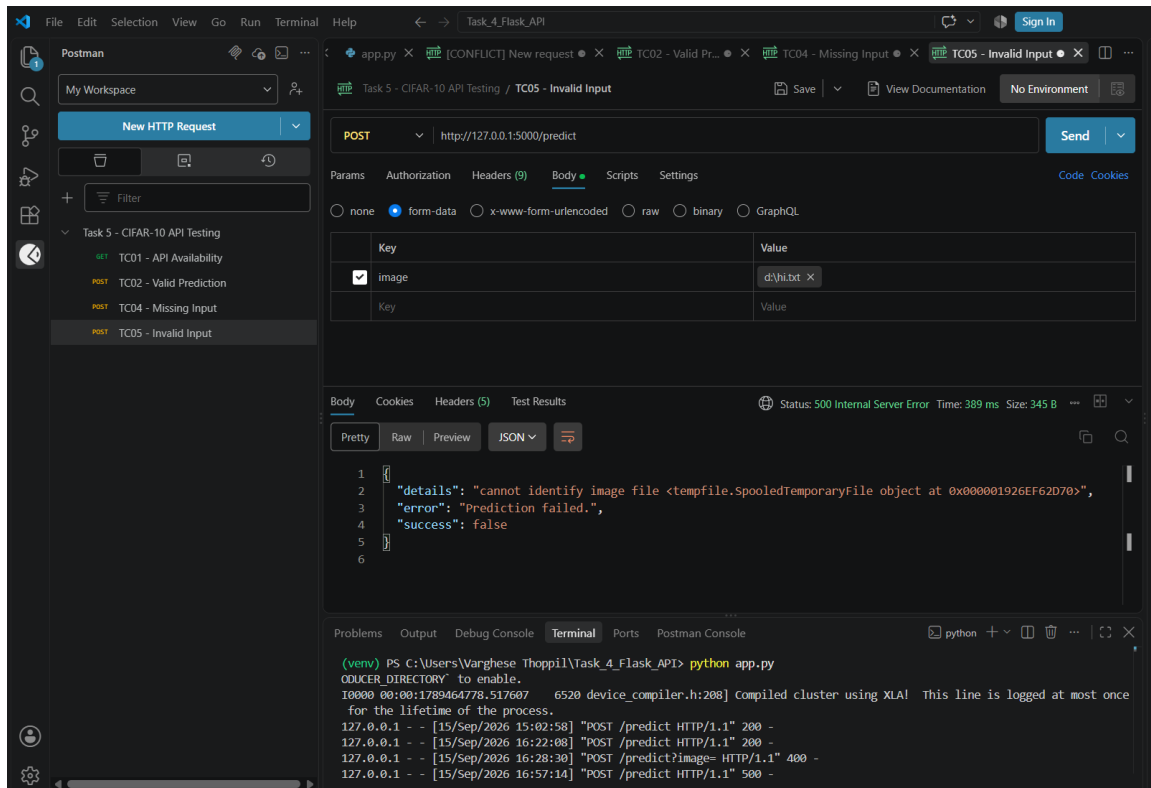


Figure 7 — TC05: a text file submitted in the image field returning 500 Internal Server Error in 389 ms with "Prediction failed." and the underlying decode error in the details field.

9.3 TC06 — Empty input

What was intentionally incorrect: a POST to `/predict` with no query parameters and no body at all — the minimal malformed request.

Observed response: HTTP 400 Bad Request in 6 ms, 275 bytes, with a body identical to TC04 ("No image provided." / "Please upload an image using the 'image' field. "). The access log records POST `/predict` HTTP/1.1 400 at 17:18:46.

Assessment: error handling worked as observed, and is consistent with TC04. Two different ways of omitting the image — an empty query parameter and no body whatsoever — converge on the same status code, the same response size (275 B) and the same message, which indicates a single, well-placed input-presence check rather than ad-hoc handling.

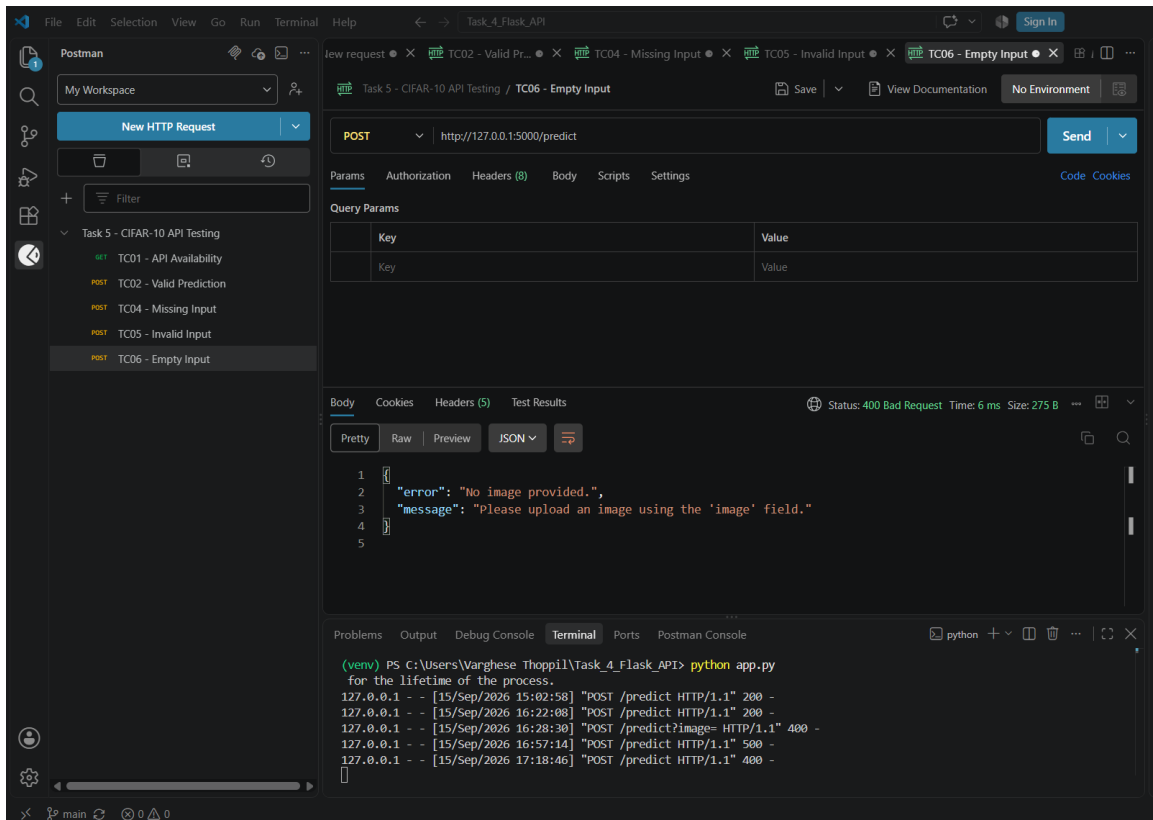


Figure 8 — TC06: POST to /predict with no parameters and no body returning 400 Bad Request in 6 ms with the same error payload as TC04.

9.4 TC07 — Invalid endpoint

What was intentionally incorrect: a GET request to `http://127.0.0.1:5000/invalid`, a route the API does not define.

Observed response: HTTP 404 Not Found in 10 ms, 251 bytes:

```
{
  "error": "Endpoint not found.",
  "message": "Please check the API URL."
}
```

The access log records GET /invalid HTTP/1.1" 404 at 17:22:43.

Assessment: error handling worked as observed. Flask's default behaviour for an unmatched route is an HTML error page; the JSON body returned here shows that a custom 404 handler has been registered, so a client parsing responses as JSON will not encounter an unexpected content type when it mistypes a URL.

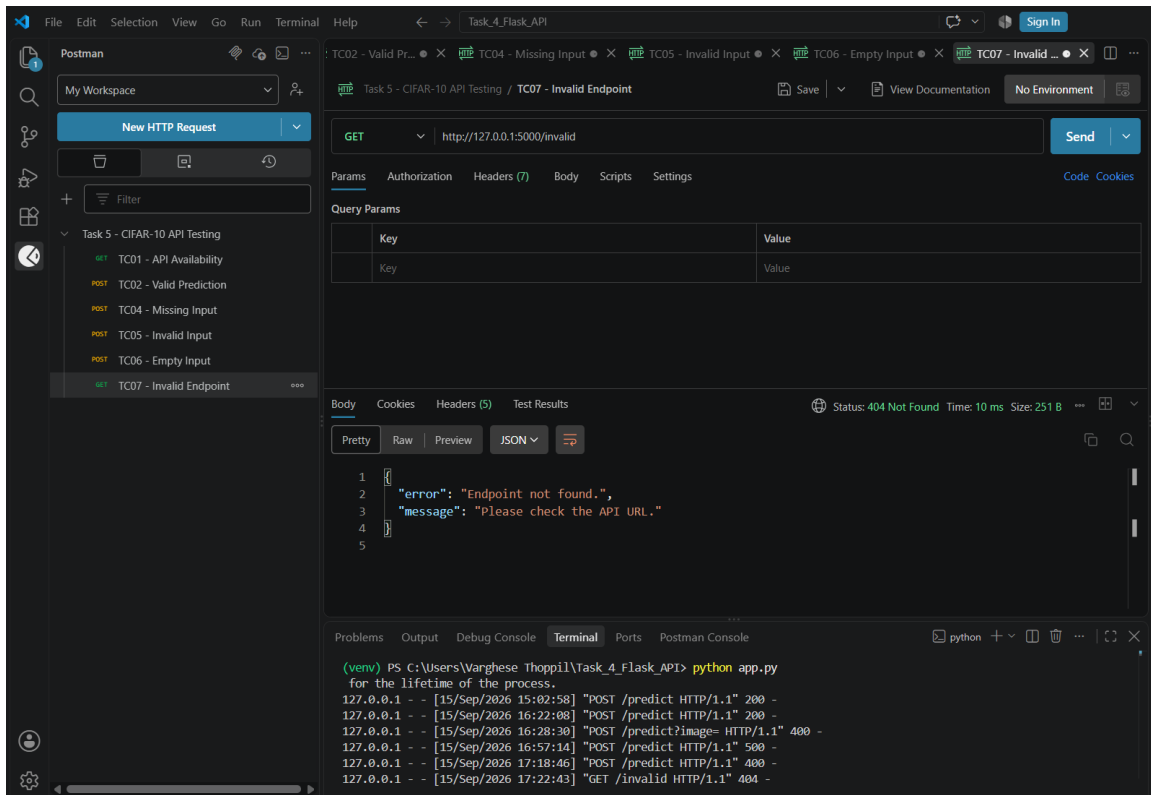


Figure 9 — TC07: GET to the undefined route `/invalid` returning 404 Not Found in 10 ms with a JSON error body rather than an HTML error page.

10. Curl Testing

Two Curl tests were performed from a PowerShell session in the project directory, providing an independent cross-check of the Postman results.

10.1 Root endpoint via Curl

Command as visible in the evidence:

```
curl http://127.0.0.1:5000/
```

An important observation about this invocation: the output is not the plain response body that `curl.exe` produces, but a PowerShell object listing with the fields `StatusCode`, `StatusDescription`, `Content`, `RawContent`, `Forms`, `Headers`, `Images`, `InputFields`, `Links`, `ParsedHtml` and `RawContentLength`. In Windows PowerShell, `curl` is an alias for the `Invoke-WebRequest` cmdlet, so this command exercised the cmdlet rather than Curl itself. The screenshot also shows a confirmation prompt ("Do you want to continue?") answered with `Y` before the request proceeded.

Observed result: `StatusCode 200`, `StatusDescription OK`, and a `Content` field carrying the same JSON as the Postman TC01 response. The `RawContent` section additionally exposes the response headers:

```

HTTP/1.1 200 OK
Connection: close
Content-Length: 171
Content-Type: application/json
Date: Tue, 15 Sep 2026 12:05:14 GMT
Server: Werkzeug/3.1.8 Python/3.12.8

```

These headers are testing evidence in their own right: they confirm the declared content type of application/json, identify the server stack as the Werkzeug development server on Python 3.12.8, and give a payload length of 171 bytes.

```
(venv) PS C:\Users\Varghese Thoppil\Task_4_Flask_API> curl http://127.0.0.1:5000/

Do you want to continue?

[Y] Yes [A] Yes to All [N] No [L] No to All [S] Suspend [?] Help (default is "N"): Y

StatusCode      : 200
StatusDescription : OK
Content          : {
  "endpoint": "/predict",
  "message": "CIFAR-10 Deep Learning Prediction API",
  "method": "POST",
  "model": "cifar10_cnn_model.keras",
  "status": "API is running"
}

RawContent      : HTTP/1.1 200 OK
                  Connection: close
                  Content-Length: 171
                  Content-Type: application/json
                  Date: Tue, 15 Sep 2026 12:05:14 GMT
                  Server: Werkzeug/3.1.8 Python/3.12.8

                  {
                    "endpoint": "/predict",
                    "mes...
Forms           : {}
Headers         : {[Connection, close], [Content-Length, 171], [Content-Type, application/json],
                  [Date, Tue, 15 Sep 2026 12:05:14 GMT]...}
Images          : {}
InputFields     : {}
Links           : {}
ParsedHtml      : mshtml.HTMLDocumentClass
RawContentLength : 171
```

Figure 10 — Root endpoint accessed from PowerShell. The object-style output (StatusCode, RawContent, ParsedHtml) shows that curl resolved to the Invoke-WebRequest alias; the response headers confirm Content-Type: application/json and Server: Werkzeug/3.1.8 Python/3.12.8.

10.2 Prediction endpoint via Curl

Command as visible in the evidence:

```
curl.exe -X POST -F "image=@C:\Users\Varghese Thoppil\Task_4_Flask_API\test\sample_images\test_image.jpg"
http://127.0.0.1:5000/predict
```

Here curl.exe is invoked explicitly, bypassing the PowerShell alias, so this test genuinely exercises Curl. The request is a POST to /predict with -F constructing a multipart/form-data body whose field name is image and whose value is the file test_image.jpg, identical in field name and file to the Postman TC02 and TC03 requests.

Observed result: the raw JSON prediction payload was printed to the console — class_index 5, confidence 99.99, prediction "dog", the full ten-class probabilities object and success true — matching the Postman response of Figure 5 value for value, including the full floating-point precision of every probability. The shell returned to the prompt without error.

Note: because curl.exe writes only the response body by default, the HTTP status code is not visible in this capture. Success is inferred from the well-formed prediction payload and from the Flask access log lines recording POST /predict with status 200. No -i or -v flag was used, so the status line for this specific invocation cannot be read directly from the evidence.

```

(venv) PS C:\Users\Varghese Thoppil\Task_4_Flask_API> curl.exe -X POST -F "image=@C:\Users\Varghese Thoppil\Task_4_Flask_API\test\sample_images\test_image.jpg" http://127.0.0.1:5000/predict
{
  "class_index": 5,
  "confidence": 99.99,
  "prediction": "dog",
  "probabilities": {
    "airplane": 1.986864232167429e-12,
    "automobile": 9.628090527266913e-15,
    "bird": 1.2026609510940034e-05,
    "cat": 8.43729212647304e-05,
    "deer": 8.452229888211704e-11,
    "dog": 0.9999029636383057,
    "frog": 3.92518274827669e-11,
    "horse": 5.793619948235573e-07,
    "ship": 2.0803866103229728e-15,
    "truck": 2.1446757672745342e-13
  },
  "success": true
}
(venv) PS C:\Users\Varghese Thoppil\Task_4_Flask_API>

```

Figure 11 — Prediction request issued with `curl.exe` using `-F "image=@..."`. The returned JSON matches the Postman response exactly, confirming client-independent behaviour.

11. Screenshots and Test Evidence

The screenshot shows the Postman interface with a new request configured as a GET to `http://127.0.0.1:5000/`. The response is a 200 OK status with a response time of 12 ms and a size of 337 B. The response body is displayed in JSON format:

```

{
  "endpoint": "/predict",
  "message": "CIFAR-10 Deep Learning Prediction API",
  "method": "POST",
  "model": "cifar10_cnn_model.keras",
  "status": "API is running"
}

```

Below the Postman interface, an integrated terminal shows the Flask application startup logs and access logs:

```

(venv) PS C:\Users\Varghese Thoppil\Task_4_Flask_API> python
Instructions in performance-critical operations.
To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX AVX2 AVX_VNNI FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Please use WSL2 or the TensorFlow-DirectML plugin.
Model loaded successfully.
* Debugger is active!
* Debugger PIN: 572-980-906
127.0.0.1 - - [15/Sep/2026 14:34:29] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [15/Sep/2026 14:50:44] "GET / HTTP/1.1" 200 -

```

Figure 1 — TC01: GET `http://127.0.0.1:5000/` returning 200 OK in 12 ms, with the Flask startup log and access log visible in the integrated terminal.

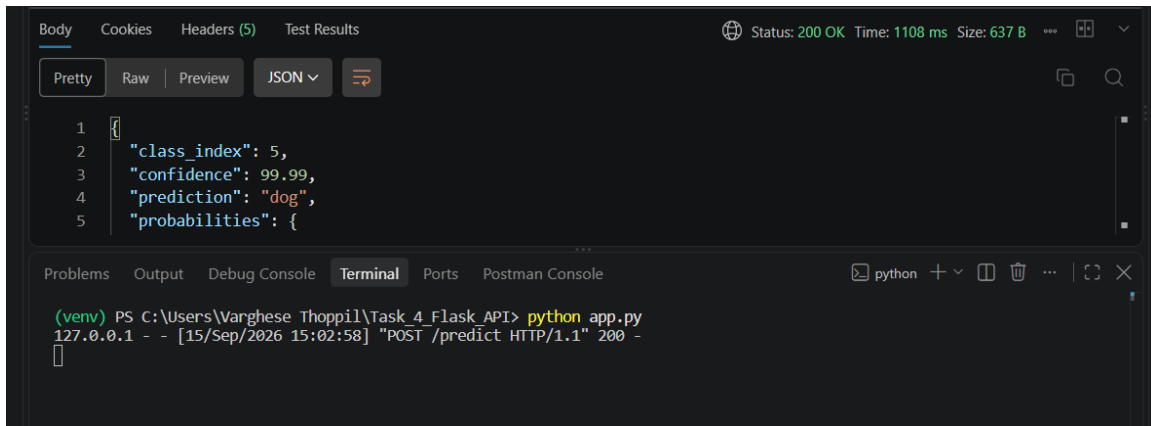


Figure 2 — TC02: 200 OK in 1108 ms for the valid prediction request, with the corresponding Flask access log entry below.



Figure 3 — The input image test_image.jpg used for TC02, TC03 and the Curl prediction test. The API classified this image as "dog".

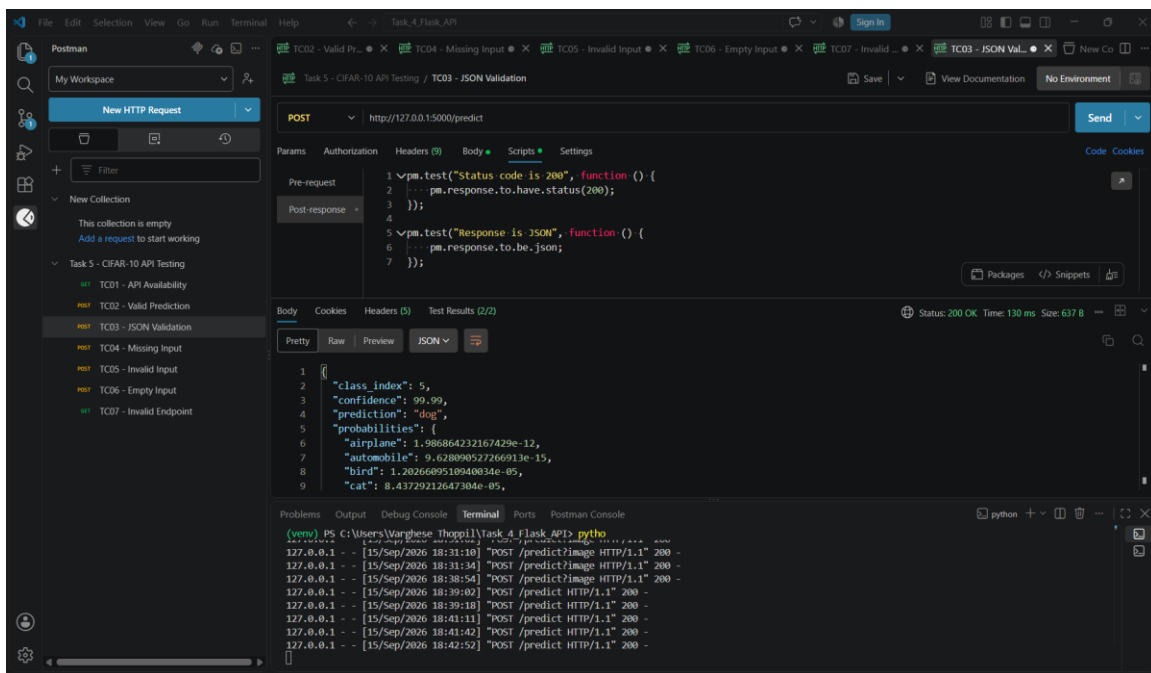


Figure 4 — TC03: the Postman post-response script (upper pane), the 200 OK / 130 ms / 637 B result with Test Results (2/2), and the accumulated Flask access log (lower pane).

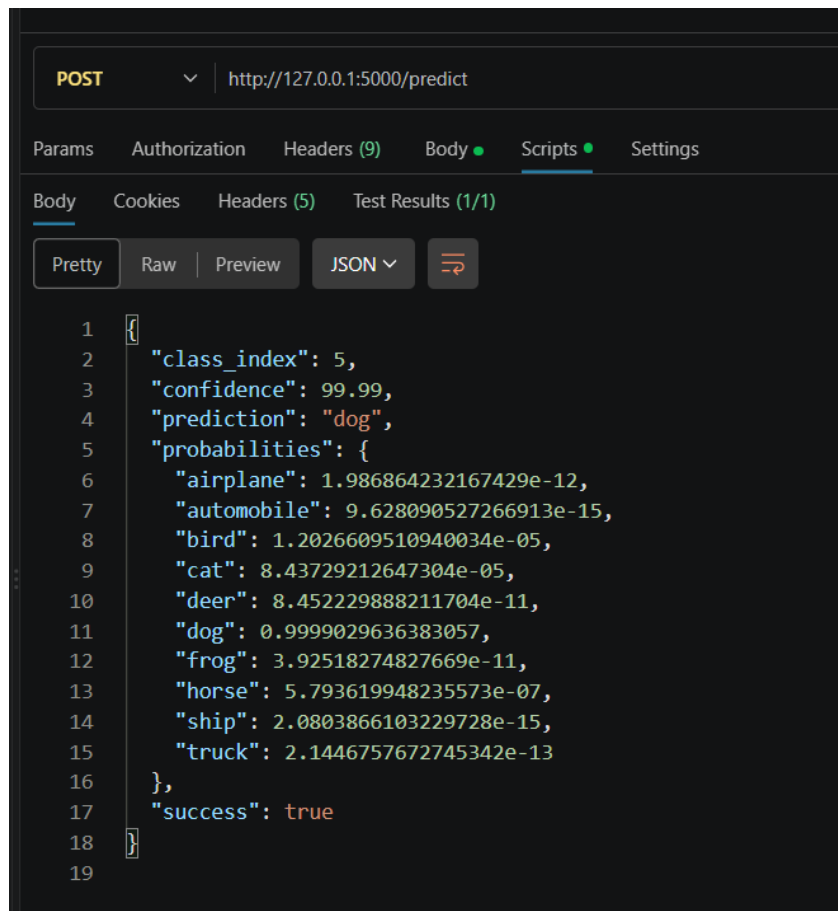


Figure 5 — Complete JSON prediction response showing all five top-level keys and the ten per-class probabilities. The Test Results label on this capture reads (1/1); see discrepancy D3 in Section 5.3.

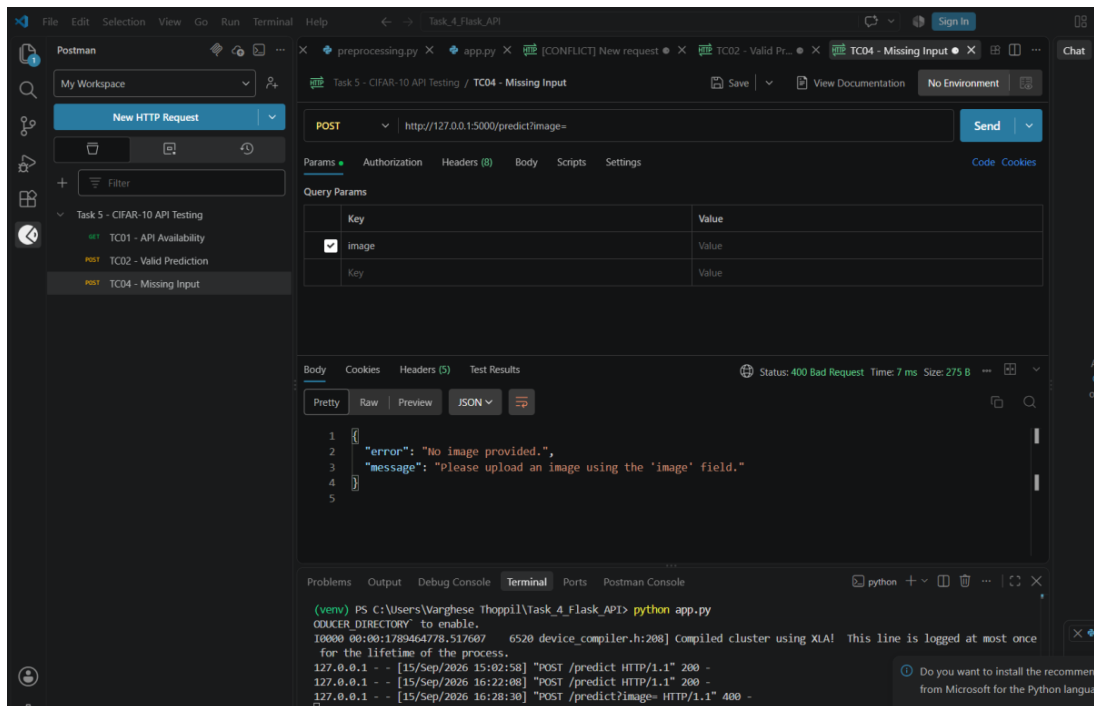


Figure 6 — TC04: POST to /predict?image= returning 400 Bad Request in 7 ms with the "No image provided." JSON error, and the matching 400 entry in the Flask access log.

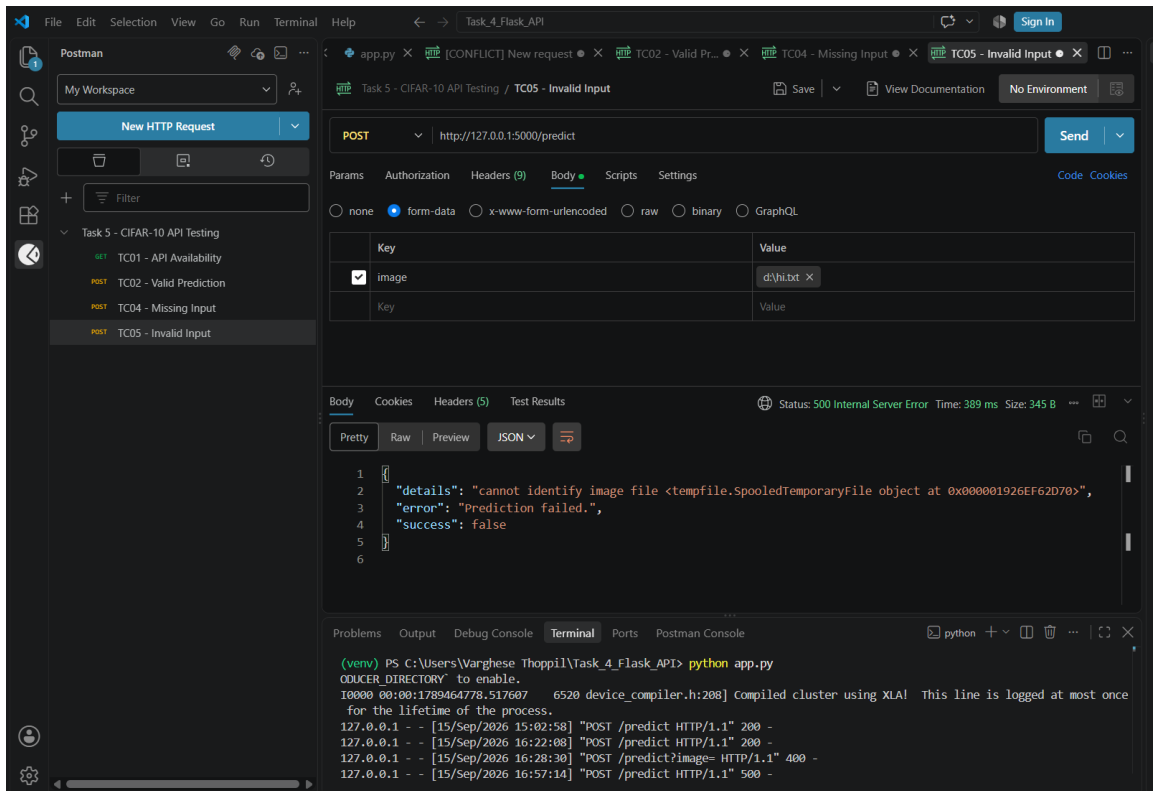


Figure 7 — TC05: a text file submitted in the image field returning 500 Internal Server Error in 389 ms with "Prediction failed." and the underlying decode error in the details field.

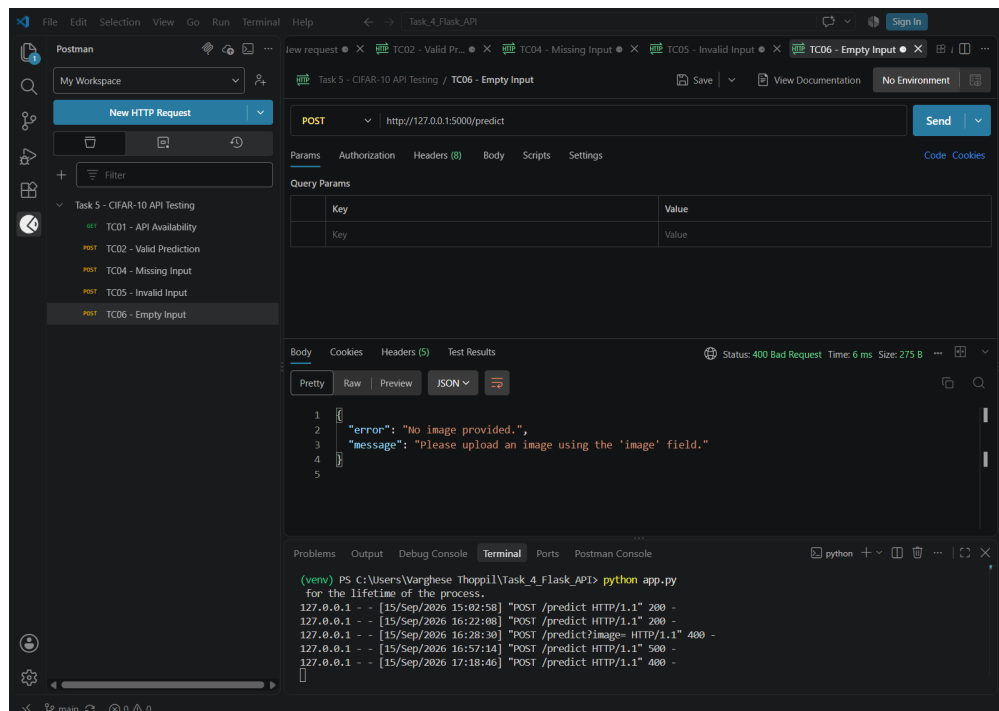


Figure 8 — TC06: POST to /predict with no parameters and no body returning 400 Bad Request in 6 ms with the same error payload as TC04.

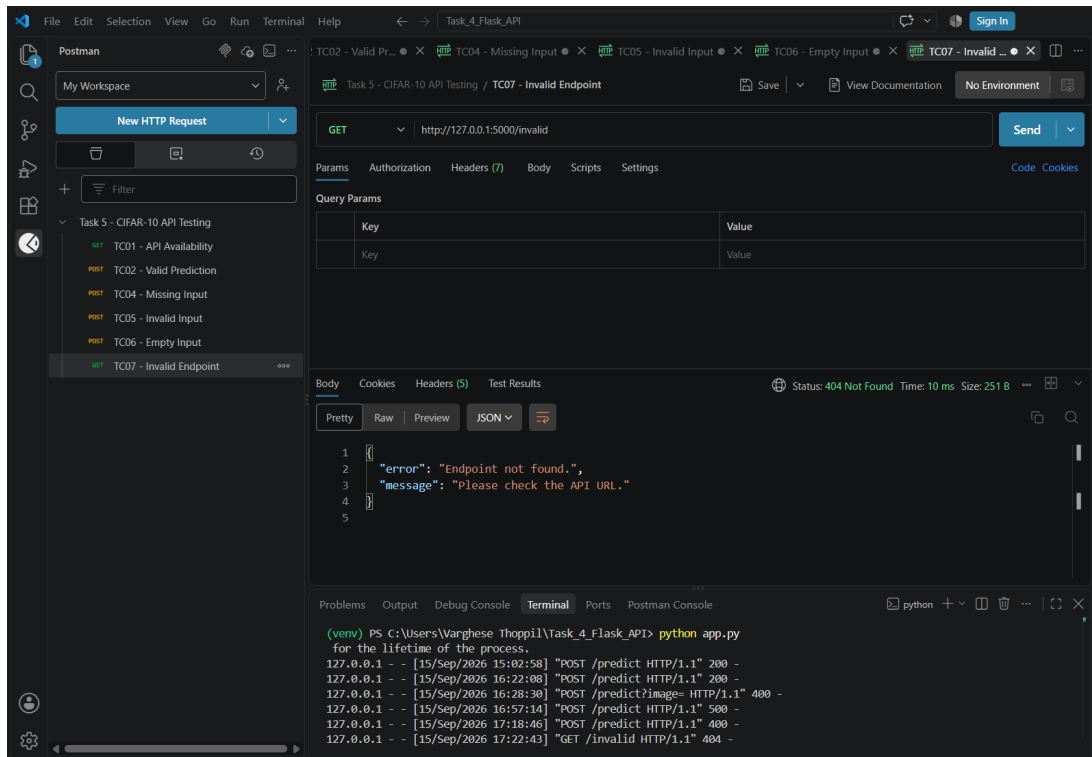


Figure 9 — TC07: GET to the undefined route /invalid returning 404 Not Found in 10 ms with a JSON error body rather than an HTML error page.

```

(venv) PS C:\Users\Varghese Thoppil\Task_4_Flask_API> curl http://127.0.0.1:5000/

Do you want to continue?

[Y] Yes [A] Yes to All [N] No [L] No to All [S] Suspend [?] Help (default is "N"): Y

StatusCode      : 200
StatusDescription : OK
Content         : {
  "endpoint": "/predict",
  "message": "CIFAR-10 Deep Learning Prediction API",
  "method": "POST",
  "model": "cifar10_cnn_model.keras",
  "status": "API is running"
}

RawContent      : HTTP/1.1 200 OK
                  Connection: close
                  Content-Length: 171
                  Content-Type: application/json
                  Date: Tue, 15 Sep 2026 12:05:14 GMT
                  Server: Werkzeug/3.1.8 Python/3.12.8

                  {
                    "endpoint": "/predict",
                    "mes...
Forms           : {}
Headers         : {[Connection, close], [Content-Length, 171], [Content-Type, application/json],
                  [Date, Tue, 15 Sep 2026 12:05:14 GMT]...}
Images          : {}
InputFields     : {}
Links           : {}
ParsedHtml      : mshtml.HTMLDocumentClass
RawContentLength : 171

```

Figure 10 — Root endpoint accessed from PowerShell. The object-style output (StatusCode, RawContent, ParsedHtml) shows that curl resolved to the Invoke-WebRequest alias; the response headers confirm Content-Type: application/json and Server: Werkzeug/3.1.8 Python/3.12.8.

```
(venv) PS C:\Users\Varghese Thoppil\Task_4_Flask_API> curl.exe -X POST -F "image=@C:\Users\Varghese Thoppil\Task_4_Flask_API\test\sample_images\test_image.jpg" http://127.0.0.1:5000/predict
{
  "class_index": 5,
  "confidence": 99.99,
  "prediction": "dog",
  "probabilities": {
    "airplane": 1.986864232167429e-12,
    "automobile": 9.628090527266913e-15,
    "bird": 1.2026609510940034e-05,
    "cat": 8.43729212647304e-05,
    "deer": 8.452229888211704e-11,
    "dog": 0.9999029636383057,
    "frog": 3.92518274827669e-11,
    "horse": 5.793619948235573e-07,
    "ship": 2.0803866103229728e-15,
    "truck": 2.1446757672745342e-13
  },
  "success": true
}
(venv) PS C:\Users\Varghese Thoppil\Task_4_Flask_API>
```

Figure 11 — Prediction request issued with curl.exe using -F "image=@...". The returned JSON matches the Postman

12. Response Logs

Two categories of log evidence are available: the response metadata reported by Postman for each request, and the Flask/Werkzeug access log visible in the integrated terminal across several screenshots.

12.1 Postman response metadata

Test case	Status	Response time	Size	Source figure
TC01	200 OK	12 ms	337 B	Figure 1
TC02	200 OK	1108 ms	637 B	Figure 2
TC03	200 OK	130 ms	637 B	Figure 4
TC04	400 Bad Request	7 ms	275 B	Figure 6
TC05	500 Internal Server Error	389 ms	345 B	Figure 7
TC06	400 Bad Request	6 ms	275 B	Figure 8
TC07	404 Not Found	10 ms	251 B	Figure 9

12.2 Flask access log

The Werkzeug access log accumulates across the screenshots. The sequence below is transcribed from the terminal panes of Figures 1, 6, 7, 8 and 9, and corresponds to the main test run:

```
127.0.0.1 -- [15/Sep/2026 14:34:29] "GET / HTTP/1.1" 200 -
127.0.0.1 -- [15/Sep/2026 14:50:44] "GET / HTTP/1.1" 200 -
127.0.0.1 -- [15/Sep/2026 15:02:58] "POST /predict HTTP/1.1" 200 -
127.0.0.1 -- [15/Sep/2026 16:22:08] "POST /predict HTTP/1.1" 200 -
127.0.0.1 -- [15/Sep/2026 16:28:30] "POST /predict?image= HTTP/1.1" 400 -
127.0.0.1 -- [15/Sep/2026 16:57:14] "POST /predict HTTP/1.1" 500 -
```

```
127.0.0.1 -- [15/Sep/2026 17:18:46] "POST /predict HTTP/1.1" 400 -
127.0.0.1 -- [15/Sep/2026 17:22:43] "GET /invalid HTTP/1.1" 404 -
```

This sequence corroborates the client-side results one for one: two successful health checks, two successful predictions, then the 400, 500, 400 and 404 responses of the four negative tests, in the order TC04, TC05, TC06, TC07.

A later block of log lines is visible in Figure 4, recorded after the negative tests during repeated runs of the prediction request:

```
127.0.0.1 -- [15/Sep/2026 18:31:10] "POST /predict?image HTTP/1.1" 200 -
127.0.0.1 -- [15/Sep/2026 18:31:34] "POST /predict?image HTTP/1.1" 200 -
127.0.0.1 -- [15/Sep/2026 18:38:54] "POST /predict?image HTTP/1.1" 200 -
127.0.0.1 -- [15/Sep/2026 18:39:02] "POST /predict HTTP/1.1" 200 -
127.0.0.1 -- [15/Sep/2026 18:39:18] "POST /predict HTTP/1.1" 200 -
127.0.0.1 -- [15/Sep/2026 18:41:11] "POST /predict HTTP/1.1" 200 -
127.0.0.1 -- [15/Sep/2026 18:41:42] "POST /predict HTTP/1.1" 200 -
127.0.0.1 -- [15/Sep/2026 18:42:52] "POST /predict HTTP/1.1" 200 -
```

Three of these entries carry the path `/predict?image` while returning 200. This differs from TC04, where the path `/predict?image=` returned 400. The distinction is that a query string may be present on the URL without affecting the outcome, provided a valid multipart file part is also supplied — the earlier 400 arose from the absence of the file part, not from the presence of the query string. This reading is consistent with all the evidence available, but no screenshot captures the client side of these particular requests, so it is presented as an interpretation of the log rather than as a separately executed and verified test case.

No response body is written to the server log; the full response payloads are available only from the client-side captures in Figures 1, 2 and 5–11.

13. Test Results Summary

Test case	Result	HTTP status	Validation performed	Observation
TC01 API Availability	Pass	200 OK	Status line; presence of status, endpoint, method and model fields.	Service reachable; model <code>cifar10_cnn_model.keras</code> reported as loaded.
TC02 Valid Prediction	Pass	200 OK	Status line; prediction payload fields and values.	Correct prediction returned; 1108 ms, the slowest successful call.
TC03 JSON Validation	Pass	200 OK	Automated: <code>pm.test</code> status-200 and is-JSON assertions, 2/2 passed.	Same payload as TC02 at 130 ms.
TC04 Missing Input	Pass	400 Bad Request	Status class; error and message fields.	Query parameter correctly rejected as a substitute for a file part.
TC05 Invalid Input	Partial	500 Internal Server Error	Status class; error, details and success fields.	Failure contained and returned as JSON, but a client-input fault reported with a server-error status; internal exception text exposed.
TC06 Empty Input	Pass	400 Bad Request	Status class; error and message fields.	Identical handling to TC04.
TC07 Invalid Endpoint	Pass	404 Not Found	Status class; JSON error body instead of an HTML page.	Custom JSON 404 handler confirmed.
Curl Root endpoint	Pass	200 OK	StatusCode, Content-Type and response headers.	Executed via the PowerShell <code>Invoke-WebRequest</code> alias rather than <code>curl.exe</code> .

Test case	Result	HTTP status	Validation performed	Observation
Curl Prediction	Pass	Not shown in the capture	Response body compared field by field against the Postman payload.	Byte-equivalent to the Postman result; status inferred from the payload and the access log.

Outcome: six of the seven Postman test cases fully satisfied their validation criteria; TC05 is recorded as partial for the reasons given in Section 9.2. Both Curl checks returned the expected content. No test case failed outright in the sense of the service becoming unavailable or returning an unparseable response.

14. Observations

14.1 API availability

The service started successfully, loaded the Keras model (the startup log in Figure 1 shows "Model loaded successfully."), and remained available throughout the test session — the access log spans from 14:34 to 18:42 on the same day without a restart being visible between test cases. The root endpoint responded in 12 ms. The service ran on the Werkzeug development server, which is appropriate for the scope of this task but is not intended for production use.

14.2 Valid prediction behaviour

The API returned a correct and highly confident classification for the supplied test image, identifying it as "dog" with a probability of 0.9999. Visual inspection of test_image.jpg (Figure 3) is consistent with that label. It should be noted that classification accuracy was measured against exactly one image in this task; Task 5 is concerned with API behaviour rather than model evaluation, and no accuracy claim beyond this single observation is made here.

14.3 Response time

The first prediction (TC02) took 1108 ms while the second (TC03), sending the same file to the same endpoint, took 130 ms — roughly one-eighth of the time. This pattern is characteristic of first-call overhead in TensorFlow-backed services, where the initial inference triggers graph construction and kernel initialisation that subsequent calls reuse. The evidence supports the observation of the timing difference; the attribution to warm-up is offered as a technical explanation, since no profiling was performed.

14.4 JSON response validation

All nine observed responses — successes and errors alike — were valid JSON, and the root endpoint's headers confirm Content-Type: application/json. The prediction payload is well structured and internally consistent. A limitation of the automated validation should be recorded: the collection's two assertions cover the status code and the content type only. A client integrating against this API would benefit from assertions on the presence of prediction, confidence and success, and on probabilities containing ten entries. Such assertions were not defined in the supplied collection and no claim is made that they were executed.

14.5 Negative testing and error handling

Error handling is present and deliberate rather than incidental. The API distinguishes three separate failure modes and returns a distinct, human-readable message for each. The two missing-input variants converge on the same 400 response, indicating a single input-presence check. The custom JSON 404 handler prevents content-type surprises on mistyped URLs.

The one substantive finding is TC05. A non-image file in a correctly formed request produces a 500 Internal Server Error carrying the raw decoding exception. The service recovered and continued serving requests, so this is not a stability defect, but two improvements would follow from it: mapping decode failures onto a 400-class status, and suppressing the internal exception text from the client-facing body while logging it server-side.

A secondary finding is that the error schema is not uniform. The 400 and 404 responses use error and message; the 500 response uses error, details and success. The success flag appears in the 200 and 500 responses but is absent from the 400 and 404 responses. A client cannot therefore rely on success being present to determine the outcome of a call and must inspect the HTTP status code instead.

14.6 Issues observed during testing

- The exported Postman collection is incomplete: five of the seven requests were saved with an empty URL and no body configuration, even though the screenshots show those requests being executed with full configurations. The export therefore cannot be re-run as-is to reproduce the documented results. This is a collection-hygiene issue in the testing artefact, not a defect in the API.
- One evidence capture (Figure 5) reports Test Results (1/1) where the collection defines two assertions and Figure 4 reports (2/2). The capture is cropped and cannot be attributed to a named request, so the discrepancy is recorded rather than resolved.
- The first Curl test invoked PowerShell's Invoke-WebRequest alias rather than curl.exe. The response content it returned is valid evidence of API behaviour, but only the second test (Figure 11) is a genuine Curl invocation.
- No -i or -v flag was used in the Curl prediction test, so the HTTP status line for that request is not directly visible in the evidence.
- Testing was confined to a single local instance with a single test image. No concurrency, payload-size, unsupported-method, or malformed-multipart testing is evidenced, and none is claimed.

15. Conclusion

The CIFAR-10 Flask prediction API developed in Task 4 was tested using Postman and Curl across seven test cases covering availability, valid prediction, JSON response validation and four negative scenarios. The testing confirmed, on the evidence available, that:

- The service is available at <http://127.0.0.1:5000/> and reports its status, prediction endpoint, accepted method and loaded model through a self-documenting root endpoint.
- A valid multipart/form-data request with an image file returns HTTP 200 and a well-formed JSON payload containing the predicted class, class index, confidence and a complete ten-class probability distribution.
- The two Postman assertions defined for TC03 — status code 200 and a JSON response — both passed.
- Requests with no image file are rejected with HTTP 400 and an explanatory JSON message, consistently across two different ways of omitting the input.
- An undefined route returns HTTP 404 with a JSON error body rather than an HTML error page.
- Curl independently reproduced both the root-endpoint response and the prediction payload exactly, confirming that the behaviour is not specific to the Postman client.

Not all tests satisfied their validation criteria without qualification. TC05 showed that a non-image file submitted in the image field produces HTTP 500 with the underlying Python decoding exception exposed in the response body. The service handled the condition without crashing and returned structured JSON, but the status class does not reflect the client-side origin of the fault and the internal error text should not be surfaced to clients. This is recorded as the principal finding of the exercise.

It is also recorded that the supplied Postman collection export is incomplete — five of seven requests lack their URL and body — so the collection as exported does not reproduce the documented run without reconfiguration. The results in this report rest on the screenshot evidence, which is corroborated throughout by the Flask access log.

Overall, the API demonstrated stable availability, a consistent and machine-readable success contract, and deliberate error handling across the scenarios tested, with one status-code mapping issue and one error-schema inconsistency identified for correction. No claim is made that the API is defect-free or that all tests passed without qualification; the testing was limited to a single local instance, a single sample image and the seven scenarios documented above.

16. References

The references below correspond to tools, specifications and technologies directly evidenced in the testing material.

#	Reference	Relevance
1	Postman Collection Format v2.1.0 schema — https://schema.getpostman.com/json/collection/v2.1.0/collection.json	Declared in the info.schema field of the supplied collection export.
2	Postman Documentation — writing tests with the pm API (pm.test, pm.response)	Basis of the two assertions in TC03.
3	Curl project documentation — the -X and -F options	Options used in the Curl prediction test in Figure 11.
4	Microsoft PowerShell documentation — Invoke-WebRequest	Explains the object-style output in Figure 10, where curl resolves to this cmdlet alias.
5	Flask and Werkzeug documentation (Werkzeug 3.1.8, reported in the Server response header)	Framework hosting the API under test.
6	RFC 9110 — HTTP Semantics (status code classes 2xx, 4xx, 5xx)	Basis for the status-code assessment in Sections 9.2 and 14.5.
7	RFC 7578 — Returning Values from Forms: multipart/form-data	Format of the prediction request body.
8	The CIFAR-10 dataset (Krizhevsky, 2009) — ten-class image classification benchmark	Source of the ten class labels returned in the probabilities object.
9	Task 4 project artefacts: app.py and cifar10_cnn_model.keras, directory Task_4_Flask_API	The application under test, as named in the evidence. The source code was not supplied for Task 5 and was not inspected.

17. Source Code

app.py

```
from flask import Flask, request, jsonify
```

```

from tensorflow.keras.models import load_model
from PIL import Image
import numpy as np
import os

from utils.preprocessing import preprocess_image, CLASS_NAMES

# -----
# Flask Application
# -----

app = Flask(__name__)

# -----
# Model Configuration
# -----

MODEL_PATH = os.path.join(
    os.path.dirname(__file__),
    "model",
    "cifar10_cnn_model.keras"
)

# -----
# Load Trained Deep Learning Model
# -----

try:
    model = load_model(MODEL_PATH)
    print("Model loaded successfully.")
except Exception as e:
    model = None
    print(f"Error loading model: {e}")

# -----
# Home Endpoint
# -----

@app.route("/", methods=["GET"])
def home():
    return jsonify({
        "message": "CIFAR-10 Deep Learning Prediction API",
        "status": "API is running",
        "model": "cifar10_cnn_model.keras",
    })

```

```

        "endpoint": "/predict",
        "method": "POST"
    })

# -----
# Health Check Endpoint
# -----

@app.route("/health", methods=["GET"])
def health():
    if model is not None:
        return jsonify({
            "status": "healthy",
            "model_loaded": True
        }), 200

    return jsonify({
        "status": "unhealthy",
        "model_loaded": False
    }), 500

# -----
# Prediction Endpoint
# -----

@app.route("/predict", methods=["POST"])
def predict():

    # Check whether model loaded
    if model is None:
        return jsonify({
            "error": "Model could not be loaded."
        }), 500

    # Check whether image is present
    if "image" not in request.files:
        return jsonify({
            "error": "No image provided.",
            "message": "Please upload an image using the 'image' field."
        }), 400

    file = request.files["image"]

    # Check filename
    if file.filename == "":
        return jsonify({

```

```

        "error": "No image selected."
    }), 400

try:

    # Open image
    image = Image.open(file.stream)

    # Preprocess image
    processed_image = preprocess_image(image)

    # Generate prediction
    predictions = model.predict(processed_image, verbose=0)

    # Get predicted class
    predicted_index = int(np.argmax(predictions[0]))

    predicted_class = CLASS_NAMES[predicted_index]

    # Get confidence
    confidence = float(predictions[0][predicted_index])

    # Get all class probabilities
    probabilities = {
        CLASS_NAMES[i]: float(predictions[0][i])
        for i in range(len(CLASS_NAMES))
    }

    return jsonify({
        "success": True,
        "prediction": predicted_class,
        "class_index": predicted_index,
        "confidence": round(confidence * 100, 2),
        "probabilities": probabilities
    }), 200

except Exception as e:

    return jsonify({
        "success": False,
        "error": "Prediction failed.",
        "details": str(e)
    }), 500

# -----
# Error Handlers
# -----

```

```

@app.errorhandler(404)
def not_found(error):
    return jsonify({
        "error": "Endpoint not found.",
        "message": "Please check the API URL."
    }), 404

@app.errorhandler(405)
def method_not_allowed(error):
    return jsonify({
        "error": "Method not allowed.",
        "message": "Please use the correct HTTP method."
    }), 405

@app.errorhandler(500)
def internal_server_error(error):
    return jsonify({
        "error": "Internal server error."
    }), 500

# -----
# Run Flask Application
# -----

if __name__ == "__main__":
    app.run(
        host="127.0.0.1",
        port=5000,
        debug=True
    )

```

Task 5 - CIFAR-10 API Testing.postman collection.json

```

{
  "info": {
    "_postman_id": "245b12f3-7691-4d93-a004-baa86b084177",
    "name": "Task 5 - CIFAR-10 API Testing",
    "schema": "https://schema.getpostman.com/json/collection/v2.1.0/collection.json",
    "_exporter_id": "41861669"
  },
  "item": [
    {

```

```

    "name": "TC01 - API Availability",
    "request": {
      "method": "GET",
      "header": [],
      "url": {
        "raw": ""
      }
    },
    "response": []
  },
  {
    "name": "TC02 - Valid Prediction",
    "request": {
      "method": "POST",
      "header": [],
      "body": {
        "mode": "formdata",
        "formdata": [
          {
            "key": "image",
            "type": "file",
            "id": 0,
            "fileUploadError": "",
            "src": [
              "c:\\Users\\Varghese
Thoppil\\Task_4_Flask_API\\test\\sample_images\\test_image.jpg"
            ]
          }
        ]
      }
    },
    "url": {
      "raw": "http://127.0.0.1:5000/predict",
      "protocol": "http",
      "host": [
        "127",
        "0",
        "0",
        "1"
      ],
      "port": "5000",
      "path": [
        "predict"
      ]
    }
  },
  "response": []
},
{

```

```

"name": "TC03 - JSON Validation",
"event": [
  {
    "listen": "test",
    "script": {
      "exec": [
        "pm.test(\"Status code is 200\", function () {\r",
        "  pm.response.to.have.status(200);\r",
        "});\r",
        "\r",
        "pm.test(\"Response is JSON\", function () {\r",
        "  pm.response.to.be.json;\r",
        "});\r",
        "];
      "type": "text/javascript",
      "packages": {}
    }
  }
],
"request": {
  "method": "POST",
  "header": [],
  "body": {
    "mode": "formdata",
    "formdata": [
      {
        "key": "image",
        "type": "file",
        "id": 2,
        "fileUploadError": "",
        "src": [
          "c:\\Users\\Varghese
Thoppil\\Task_4_Flask_API\\test\\sample_images\\test_image.jpg"
        ]
      }
    ]
  }
},
"url": {
  "raw": "http://127.0.0.1:5000/predict",
  "protocol": "http",
  "host": [
    "127",
    "0",
    "0",
    "1"
  ],
  "port": "5000",
  "path": [

```

```

        "predict"
      ]
    }
  },
  "response": []
},
{
  "name": "TC04 - Missing Input",
  "request": {
    "method": "POST",
    "header": [],
    "url": {
      "raw": ""
    }
  },
  "response": []
},
{
  "name": "TC05 - Invalid Input",
  "request": {
    "method": "POST",
    "header": [],
    "url": {
      "raw": ""
    }
  },
  "response": []
},
{
  "name": "TC06 - Empty Input",
  "request": {
    "method": "POST",
    "header": [],
    "url": {
      "raw": ""
    }
  },
  "response": []
},
{
  "name": "TC07 - Invalid Endpoint",
  "request": {
    "method": "GET",
    "header": [],
    "url": {
      "raw": ""
    }
  },
  "response": []
},

```

```
"response": []  
  }  
]  
}
```